

Week 6 Assignment

SQL Server Request Management Application

Terminal-based Python assignment using pyodbc

Recreate the request application using SQL Server for persistence and pyodbc for database access.

Scope: Python fundamentals, SQL Server, pyodbc, database connections, cursors, parameterized queries, and basic Git/GitHub usage.

1. Assignment Overview

Your task is to build a small **terminal-based Request Management Application** using Python and SQL Server.

The application will allow a user to:

- Create a new request.
- Save the request in a SQL Server database.
- View previously created requests.
- Close the application and return later without losing the saved requests.

There is no graphical user interface. All interaction should happen through the terminal.

2. Objective

The purpose of this assignment is to apply the Python concepts covered during the first week while learning how an application works with a SQL database.

You are expected to practice:

- Variables and basic Python types
- Conditions
- Loops
- Functions
- Lists and dictionaries
- Classes
- SQL Server database creation and table creation
- Database connections and cursors
- Parameterized SQL queries
- Basic Git and GitHub usage

The focus is not on building a complex application. The focus is on organizing several simple concepts into a program that works correctly from start to finish.

3. Application Requirements

When the application starts, the user should see a simple menu similar to:

```
Request Management System

1. Create Request
2. View Requests
3. Exit

Choose an option:
```

The exact wording and formatting are your choice.

3.1 Create Request

When the user selects **Create Request**, the application must ask for:

- Request title

- Request description

Each request must also contain:

- A unique integer request ID
- A status with the default value New

For example, a request may be displayed as:

```
ID: 1
Title: Laptop problem
Description: My laptop is not connecting to Wi-Fi
Status: New
```

After the request is created, the application should confirm that it was saved successfully.

3.2 Save Requests

Requests must be saved in the SQL Server database created for this assignment. The requests must not exist only while the Python program is running.

The application should work correctly the first time it is executed, when the database contains no requests yet.

3.3 View Requests

When the user selects **View Requests**, the application should retrieve and display all previously created requests.

Each request should display at least:

```
Request #1
Title: Laptop problem
Description: My laptop is not connecting to Wi-Fi
Status: New
```

If there are no requests, display a suitable message such as:

```
No requests have been created yet.
```

3.4 Exit

The application must provide an option for the user to exit normally. The application should continue showing the menu until the user chooses to exit.

4. Database Requirements

The trainee is responsible for creating and handling the SQL Server database and the request table.

- Create the database required by the application.
- Create a table for storing requests.
- Use an integer primary key for the request ID.
- Store the request title, description, and status.
- Ensure that each request has a unique ID.
- Ensure that newly created requests have the status New.

The database name, table name, column names beyond the required data, and implementation details are part of the trainee's design work. The finished application must still provide all required behavior.

5. Technical Requirements

The application must:

- Be written in Python.
- Run from the terminal.
- Use pyodbc to connect to SQL Server.
- Use SQL Server Authentication.
- Use connection and cursor context managers with with blocks.
- Use parameterized SQL queries for values supplied by the user.
- Use functions to organize the program.
- Use appropriate Python data structures such as lists and dictionaries.
- Make use of a simple class where appropriate.
- Continue running until the user chooses to exit.
- Handle incorrect menu choices without immediately crashing.
- Handle relevant database errors appropriately.

Connection settings, including authentication details, may be written directly in the code for this assignment. Managing configuration with dotenv or environment variables is outside the scope of this version.

The application must implement the Create and Read parts of CRUD. Update and Delete are not required yet.

6. SQL Injection Prevention

The application must protect user-supplied values from SQL injection.

- Use parameterized queries when inserting or retrieving data based on user input.
- Do not build SQL statements by concatenating user input into the SQL string.
- Keep the SQL command structure separate from the values supplied by the user.

The trainee is responsible for applying the appropriate pyodbc approach in the implementation.

7. Out of Scope

The following features are **not required** for this assignment:

- Graphical user interface
- Website
- Local JSON persistence
- Database ORM
- User accounts or login
- Multiple users
- APIs
- Editing requests
- Deleting requests
- Changing request status
- Networking

- Cloud storage
- dotenv or environment-variable configuration
- External Python frameworks
- Automated tests

Do not add unnecessary complexity before the required functionality works correctly.

8. Minimum Expected Application Flow

A successful application should support this scenario:

```
> python main.py

Request Management System

1. Create Request
2. View Requests
3. Exit

Choose an option: 1

Enter request title: Printer Problem
Enter request description: The office printer is not printing.

Request created successfully.

1. Create Request
2. View Requests
3. Exit

Choose an option: 2

Request #1
Title: Printer Problem
Description: The office printer is not printing.
Status: New

1. Create Request
2. View Requests
3. Exit

Choose an option: 3

Goodbye.
```

If the program is started again, **Request #1 must still exist in the SQL Server database.**

9. Acceptance Criteria

The assignment will be considered functionally complete when all of the following work:

- [x] The database can be created successfully.
- [x] The request table can be created successfully.
- [x] The application starts successfully from the terminal.
- [x] The application connects to SQL Server using pyodbc.

- [x] A user can create a request.
- [x] Every request receives a unique integer ID.
- [x] Every new request has the status New.
- [x] A request is saved to the SQL Server database.
- [x] Multiple requests can be created.
- [x] Previously created requests can be viewed.
- [x] Requests still exist after closing and reopening the application.
- [x] The program works when there are no existing requests.
- [x] An incorrect menu choice does not immediately terminate the program.
- [x] User-supplied values are handled using parameterized SQL queries.
- [x] Connection and cursor resources are handled using with blocks.
- [x] The user can exit the application normally.

10. Submission

Submit the project through GitHub.

The repository should contain:

- The Python source code
- The SQL statements or scripts required to create the database and table
- A README.md
- Any other files required to run the application

The README should briefly explain:

1. What the application does.
2. How to create the database and table.
3. How to configure the SQL Server connection.
4. How to run the application.
5. Where requests are stored.

Use Git throughout the assignment and make meaningful commits as the application develops rather than submitting the entire project as one final commit.

11. Evaluation Focus

Functionality

Does the application create, save, reload, and display requests?

Python Fundamentals

Are variables, conditions, loops, functions, collections, and classes used appropriately?

Database Design

Were the database and table created appropriately for the required application?

Database Integration

Are the SQL Server connection and cursor handled correctly?

SQL Injection Prevention

Are user-supplied values handled through parameterized queries?

Resource Management

Are connection and cursor resources managed using with blocks?

Code Organization

Is the code reasonably clear and divided into understandable parts?

Git Usage

Was the project developed and submitted using a sensible Git workflow?

The goal is a **small, complete, working application**. The trainee is expected to make the implementation and design decisions needed to satisfy the requirements.